AE 4803 AIM: Course Notes

Elizabeth Qian, Atticus Rex, Pavlos Stavrinides

2025-01-01

Table of contents

Preface **4**

1 Introduction **5**

1.1 What's in a name? Artificial intelligence, machine learning, data science, and the scope of this course. . . . 5

1.2 Regression methods: an overview . . . 7

1.2.1 Motivation and problem description . . . 7

1.2.2 Mathematical setting and problem formulation . . . 9

1.3 A foundation for learning about ML beyond parametrized regression . . . 12

1.4 Course structure and intended learning outcomes . . . 12

2 Linear Least Squares Problems **13**

2.1 Classifying regression problems as linear vs nonlinear . . . 13

2.2 Mathematical problem formulation . . . 14

2.3 Examples . . . 15

2.3.1 Example 1: Aerodynamic drag prediction . . . 15

2.3.2 Example 2: Monitoring the Maximum Displacement of a Bridge . . . 16

2.3.3 Example 3: Detecting Cardiac Arrhythmia using Health-Monitoring Devices . . . 16

2.4 Solving linear least squares problems . . . 17

2.4.1 Using Python for Least-Squares Regression . . . 18

2.5 Assessing the learned models . . . 20

2.5.1 Common Performance Metrics . . . 20

2.5.2 Cross-Validation . . . 22

2.6 Exercises . . . 22

2.7 Further reading . . . 23

References **24**

Appendices **25**

A Linear algebra review **25**

A.1 Scalars, Vectors, and Matrices . . . 25

A.1.1 Notation . . . 25

A.1.2 The Transpose . . . 26

A.1.3 Addition & subtraction . . . 26
A.2 Scalar Multiplication . . . 27
A.2.1 Scalar-Scalar Product . . . 28
A.2.2 Scalar-Vector Product . . . 28
A.2.3 Scalar-Matrix Product . . . 28
A.3 Matrix & Vector Multiplication . . . 28
A.3.1 Vector-Vector Multiplication . . . 28
A.3.2 Matrix-Vector Multiplication . . . 29
A.3.3 Matrix-Matrix Multiplication . . . 30
A.4 Vector Norms . . . 30
A.4.1 The 2-Norm . . . 31
A.4.2 The l_p-Norm . . . 31
A.4.3 The 1-Norm . . . 31
A.4.4 The Infinity-Norm . . . 31
A.5 Rank, Range/Column-Space and Null-Spaces . . . 32
A.5.1 Linear Combinations and Span . . . 32
A.5.2 Linear-Independence . . . 32
A.5.3 Rank . . . 33
A.5.4 Range/Column-Space . . . 33
A.5.5 Null-Space . . . 34
A.5.6 The Identity Matrix . . . 35
A.5.7 Matrix Inverses . . . 35
A.6 Further Reading . . . 36

Preface

This is a Quarto book.

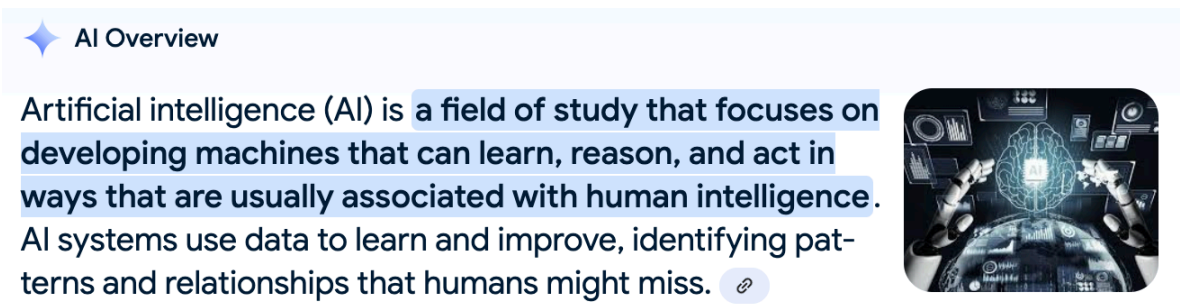
To learn more about Quarto books visit https://quarto.org/docs/books.

1 Introduction

Welcome to AE 4803 AIM: Foundations of Scientific Machine Learning. This is a new course offered in the Spring 2025 term at Georgia Tech and is intended to be a "foundations" level course in the new College of Engineering AI for Engineering minor.

1.1 What's in a name? Artificial intelligence, machine learning, data science, and the scope of this course.

I googled "artificial intelligence" (often abbreviated AI) and the first result comes from Google's new "AI overview" feature[1]:

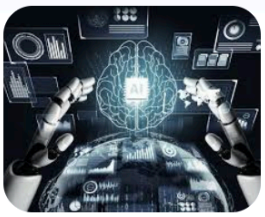


Because I closed the tab with my first search and wanted to go back to the webpage to copy the text for the image alt-text above, I repeated my search, and got a somewhat different answer:

[1] note that the constant factor $\frac{2}{N}$ drops out. It's common to play fast and loose with multiplicative constants in minimizations – other sources define the minimization objective with a $\frac{1}{2}$ multiplier, which leads to the derivative not having the factor of 2 in front and leads to the same critical point. Or you may see the objective function defined without the $1/N$ in front, which still leads to the same critical point.

AI Overview

Artificial intelligence (AI) is **the ability of a machine to perform tasks that are usually associated with human intelligence**. AI is a computer science field that studies and develops methods and software to enable machines to learn, perceive their environment, and take actions to achieve goals.



These are probably both reasonable definitions for casual conversation, but that's not what we're here for. Instead, we're here to really learn deeply about what AI is, and for that we're going to need **precision** – that is, of our definitions.

> **Exercise**
>
> Consider the two different definitions of "AI" above carefully. In what (if any) senses are they (a) exactly the same, (b) similar, (c) somewhat different, (d) very different? What consequences might these differences have (a) developing AI algorithms, (b) evaluating AI impacts, (c) creating AI policies?

I'm not going to provide a definition of AI for now and instead I'm going to throw two more terms into the mix that you may have heard. The first is "machine learning" (often abbreviated ML). Rather than get an AI definition for this too, I decided to go to the dictionary Merriam-Webster, which provides the following primary definition (Merriam-Webster Dictionary 2024):

> a computational method that is a subfield of artificial intelligence and that enables a computer to learn to perform tasks by analyzing a large dataset without being explicitly programmed

And finally I'll add the term "data science" (curiously, rarely abbreviated DS). I wanted to give you a Merriam-Webster definition here, but the term isn't in their dictionary as of writing this on October 28, 2024. So instead I'm going to use Cambridge Dictionary's definition (Cambridge Dictionary 2024):

> the use of scientific methods to obtain useful information from computer data, especially large amounts of data

> **Exercise**
>
> For both the terms "machine learning" and "data science", find an alternative definition from a source that is not a generative AI. What differences exist between the new definitions you've found and the ones I've cited above?

Clearly, different people/entities may have different ideas of what AI, ML, and data science may mean. Some people may use these terms to describe generative tools like ChatGPT, GitHub copilot, and Dall-E (or similar products developed by other entities). Others use these terms to refer to more purpose-built algorithms like AlphaGo (for playing Go) or GraphCast (for weather prediction).[2] Many academics use these terms to describe the study of the underlying mathematical and programming ideas on which such products are built.

My goal is not to give you a single definition of any of these terms and then to argue that my definition is more correct than any other definition. The point I want to make is that it's worth being clear about how we define these terms in any given context, whether it be in a textbook, a news article, or perhaps a spirited discussion between friends. To that end, I now want to make clear what it is that we will and will not cover in this class, which is a "foundations"-level course in the College of Engineering's AI minor.

The focus of this class will be the mathematical and programming foundations of *scientific machine learning*, which I define as *the study of algorithms which use scientific data to define computational tools that perform useful scientific tasks*. The main scientific task that we will focus on in this course is the task of *predictive simulation*, which seeks to predict the behavior or outcomes of scientific or engineering systems. Predictive simulation is a key task in engineering disciplines like design and control, where we seek to predict design outcomes and control responses in order to make decisions about design parameters and control inputs. The class of machine learning methods that we will focus on in this class will therefore be *regression* methods, which use data to define relationships between inputs and outputs that allow us to take a specified input and issue a prediction for the output.

1.2 Regression methods: an overview

1.2.1 Motivation and problem description

We use the notation $z \in \mathbb{R}^d$ to denote a real-valued vector of d input variables, and the notation $y \in \mathbb{R}$ to denote a real-valued scalar output variable. Our goal is to be able to predict the value of y if we know the value(s) of the input variable z. Some examples:

[2] this paragraph lists examples that came to me most quickly and thus reflects to some extent my own cognitive biases based on the media I've consumed. I welcome suggestions of other examples of AI/ML/data science to include that are less well-known – Email me!

- in aerodynamic modeling, z could contain airfoil geometry parameters like camber and thickness, and y could represent the lift coefficient. Being able to predict y from z enables engineers to choose more aerodynamically efficient designs.
- in orbital dynamics, z could represent orbital parameters like altitude and inclination, and y could represent the total time a satellite spends outside the sun's shadow in a given time period. Being able to predict y from z enables engineers to determine if a satellite's solar panels will generate enough power to support the satellite's mission.
- in chemical process engineering, z could represent conditions within a reactor like temperature and chemical mixture properties and y could represent the reaction rate. Being able to predict y from z enables engineers to design more efficient reactors.

Exercise

Propose your own example scenario where it would be useful to predict real-valued outputs from inputs, drawing on your own experience, e.g. in personal projects, previous coursework, work/internship/co-op experiences, or extracurriculars. What would z and y represent? What does predicting y from z enable in your scenario?

Mathematically, predicting y from z amounts to defining a *function* f that takes in a d-dimensional input and outputs a scalar. Mathematical notational shorthand for this is $f : \mathbb{R}^d \to \mathbb{R}$. This function f is often called a **model**. The question at hand is: how do we choose f? There are many ways to do so:

1. At a baseline, you could just make up a model: let's just say $y = f(z) = \|z\|^2$. This is mathematically valid, but it's probably a bad model for the three example scenarios above, because this model probably issues predictions that are very different from the true outputs.

2. Alternatively, you could develop a model based on physical principles (a "physics-based" model) – if you have taken classes in (aero/thermo)dynamics, then you would have learned some ways to calculate y from z in the above examples, e.g. based on potential flow theory, rigid body dynamics, or the Arrhenius equation. These models are likely to be more accurate than our made-up model above, although they are often imperfectly accurate because they make simplifying assumptions. One drawback of physics-based models is that they may be computationally expensive (some computational fluid dynamics simulations require many thousands of hours of supercomputing time). Additionally, fully physics-based models may not even exist in some applications (e.g., there are many aspects of plasma physics for which we currently lack a complete theoretical understanding).

3. Finally, our focus will be on using *data* to define a model (a "data-driven" model or a "learned" model). We assume that we have a data set consisting of N pairs of input and output data, (z_i, y_i) for $i = 1, \dots, N$. To define our model, we want to choose an f so that the predicted output $f(z_i)$ is close to the output data y_i for all the data in our data

set. This is sometimes called "fitting" the model to data. The advantages of data-driven models are that they may be significantly cheaper than physics-based models, and they can be fit to experimental data even when we lack scientific theory for developing physics-based models. The disadvantages are that data-driven models require data – and the amount of data that would be "enough" to ensure that the learned model is accurate or useful is highly dependent on the application.

Learning how to fit models to data and assess their accuracy and usefulness is the focus of this course.

1.2.2 Mathematical setting and problem formulation

Hot take: Machine learning methods, at their core, are simply very complicated calculators.

Our goal is to understand the calculations that these methods are carrying out, which is going to enable us to understand and assess both the successes and failures of the methods. This means we want to be precise about the mathematical *problem formulation*, that is, the characterization of the math problem that is being solved by the methods.

Formulating a regression problem requires three ingredients:

1. Paired input and output data: let N be the number of pairs, and let $z_1, \dots, z_N$ denote the N input data and $y_1, \dots, y_N$ denote the N corresponding outputs. It is common to notate the set of all these data pairs as $\{(z_i, y_i)\}_{i=1}^N$.
2. The definition of a parametrized *model class* – more details on this in a moment, but you should think of this as a set of possible functions that map z to y.
3. A method for choosing a model from within the chosen model class – in regression problems this is most frequently done by solving an appropriate optimization problem to pick the "best" possible model from within the class.

Let's focus on the second ingredient, the parametrized model class. We use the mathematical term "class" in a similar way to the mathematical term "set". For example, perhaps you will have heard before that the numbers $1, 2, 3, \dots$ all the way to infinity form the "set of natural numbers" (often denoted $\mathbb{N}$). As another example, $\{H, T\}$ can denote the set of possible outcomes of flipping a two-sided coin (H for heads, T for tails). Note that it is common to use curly braces $\{\cdot\}$ to enclose a set (notice where it appears in our shorthand notation for the data set!). Similarly, we can define a set of possible functions that map z to y, for example (assuming $d = 1$ so that z is a scalar for the moment):

$$\mathcal{F} := \{z^2, \sin(z), \exp(z)\}$$

In the above expression, the notation $:=$ indicates that $\mathcal{F}$ is being *defined* as the expression that follows $:=$, that is, $\mathcal{F}$ is the set of three possible functions: $f(z) = z^2$, $f(z) = \sin(z)$, and $f(z) = \exp(z)$. You see that it might be cumbersome to write the "$f(z) =$" part of every function, so that part often gets dropped.

But so far I have still only been saying "set" — what distinguishes a "class" of potential models from a "set" of potential models? Usually we use the term "class" to denote a set of functions that share certain properties or functional forms, for example (still assuming z is scalar for the moment) the class of all quadratic functions:

$$\{c_0 + c_1 z + c_2 z^2 : c_0, c_1, c_2 \in \mathbb{R}\} \tag{1.1}$$

The way to read this notation above is as the set of all functions of the form $f(z) = c_0 + c_1 z + c_2 z^2$ that can be obtained when c_0, c_1, c_2 are allowed to take on any real values.[3] We call this the *class* of all quadratic functions because by varying c_0, c_1, c_2 we always get a quadratic function (noting that with our mathematics hats on, linear functions are just special cases of quadratic functions where the second-order coefficient is zero). Thus, functions within the class share a functional form, unlike the set $\mathcal{F}$ we defined above. However, we could use the functions in the set $\mathcal{F}$ to construct an alternative *model class*:

$$\{az^2 + b\sin(z) + c\exp(z) : a, b, c \in \mathbb{R}\} \tag{1.2}$$

or, allowing z to be a d-dimensional vector again, we could consider the following model class:

$$\{\max(0, a^\top(Wz + b)) : W \in \mathbb{R}^{d\times d},\ a, b \in \mathbb{R}^d\} \tag{1.3}$$

Exercise

How would you read and understand the two classes of functions notated above?

Notice that the notation we have been using to define function classes follows a pattern: within the brackets, before the colon comes the form of the function, followed by a list of some variables that appear in the function definition, together with a specification of what values those variables are allowed to take on. We call the post-colon variables *parameters*, and the set of all values that the parameters are allowed to be chosen from is called the *parameter space*. Any of the classes of functions we have defined above can be described as a *parametrized model class*, because they define sets of functions with a shared form, where the individual functions within the set arise from varying the values of the parameters.

[3] Following the same notational convention, we could write the data set as $\{(z_i, y_i) : i = 1, \dots, N\}$, which we would read as the set of all pairs (z_i, y_i) for $i = 1, \dots, N$, but the notation $\{(z_i, y_i)\}_{i=1}^N$ is more compact and more common. I don't make the rules.

Let me describe some common notational conventions that I want you to be familiar with.[4][5] It is common to use a single lowercase letter (usually a Greek one) to denote the set of all parameters – e.g., $\beta = \{c_0, c_1, c_2\}$ or $\theta = \{a, b, W\}$, and the corresponding capital letter to denote the corresponding parameter space, i.e., $B = \mathbb{R}^3$ and $\Theta = \mathbb{R} \times \mathbb{R} \times \mathbb{R}^{d \times d}$. Where we might write $y = f(z)$ for a general abstract model f, we will typically write $y = f(z; \beta)$ or $y = f(z; \theta)$ to indicate a model parametrized by β or θ (note that we separate inputs from parameters using a semicolon).

If you've heard the term "model architecture" used to describe a machine learning model, what this means is the specification of the parametrized model class. As the term "architecture" suggests, choosing the model class is a design choice, and different model classes have varying advantages and disadvantages in terms of best possible performance, difficulty to train (more on that shortly), computational cost, and more. We'll cover many possible choices of model class, and explore their pros and cons, throughout the course.

Now we will turn our attention to the third ingredient, the method of selecting a model from within the chosen class. Because parametrized models are defined by their parameters, the choice of model amounts to choosing the parameter values. Again, you could just make up the parameter values (choose your favorite numbers), but this is unlikely to match the data well and unlikely to yield accurate predictions. In some cases you could also derive the parameter values based on physical principles (this is possible when models in the chosen class have some physical interpretation). But our focus will be on choosing parameters that define a model that "best matches" the available data. A basic and standard version of this parameter selection task is to solve the following minimization problem:

$$\theta^* = \arg\min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^{N} (f(z_i; \theta) - y_i)^2$$

Note that for the above expression to be well-defined, we need our first two ingredients – that is, we need our data set, and we need to have already chosen the parametrized model class, which gives us an expression for the parametrized function $f(z; \theta)$. Once we have those ingredients, this expression defines θ^* as the parameter that minimizes the average square error of the learned model over the available data. In this sense, this parameter θ^* defines a learned model $f(z; \theta^*)$ that "best fits" the data (it is better than all other models in its class).

[4] These are common conventions, but the ML community is comprised of a diverse group of scientists and engineers across disciplines, and it's very common to read things that use entirely different notation. This is unfortunately the reality of things, and one of the reasons I am going to push you in this class to be comfortable describing the underlying concepts precisely using different notations - that is, you are going to develop flexibility in re-assigning meaning to different symbols as we go along (I hope).

[5] I'm making an effort to explicitly define notation when I introduce it, and to give examples of how to read and understand it, but it's easy for me to miss things, so please speak up or write us a message if you have a question.

As we progress through this course, we will learn more about how we can solve the above optimization (it is different for different model architectures), and about alternative ways to select the parameter based on alternative minimization problems.

1.3 A foundation for learning about ML beyond parametrized regression

I want to emphasize that this class is not and indeed cannot (realistically) be an exhaustive introduction to machine learning, artificial intelligence, or data science. Even the community of professionals who describe themselves as working in "scientific machine learning" is quite broad in scope and would include folks working on topics that I do not intend to cover in this class. There are many topics within AI/ML/data science we will not cover (beyond perhaps at most a very surface level), including generative modeling, decision trees, unsupervised learning methods like clustering, and many more. The philosophy of this course is to empower you to learn about these other methods through future coursework or independent study, by providing a foundation in what I term the 'three pillars of artificial intelligence':

1. Mathematics: all machine learning methods are fundamentally solving mathematical problems at their core. The first pillar of AI is to be able to mathematically define the problem one seeks to solve.
2. Computation: once a mathematical problem is defined, an ML method must define an algorithm that carries out the calculations to solve the problem. The second pillar of AI is to define an algorithm that (approximately) solves the mathematical problem.
3. Human intelligence: this is the crucial third pillar of AI. Human intelligence is vital for defining mathematical problem formulations, designing algorithms, and *assessing* the results and *iteratively* updating the mathematical problem formulation and computational algorithm to achieve desired outcomes.

We will seek to develop a firm foundation in these three pillars of AI throughout the course.

1.4 Course structure and intended learning outcomes

learning outcomes

problem solving review - meant to develop your skills in demonstrating human intelligence

2 Linear Least Squares Problems

We begin our exploration of scientific machine learning methods in the fundamental setting of *linear regression problems*. The intended learning outcomes of these notes are that students should be able to:

1. Classify a regression problem as linear or nonlinear
2. Understand the abstract form of linear least squares problems
   a. Define the abstract form of linear least squares problems and be able to explain what its key ingredients are
   b. Translate a description of a model learning problem in words into a specific instance of the abstract form
   c. Construct examples of linear least squares problems, cast them in the abstract form, and explain their context
3. Solve linear least-squares problems by deriving and solving the normal equations.
4. Evaluate linear regression models by computing and interpreting mean error, mean relative error, and R^2 values on both training and test data sets.

2.1 Classifying regression problems as linear vs nonlinear

We have previously introduced regression problems in a general way: recall that the three ingredients of (parametrized) regression problems are (1) paired input and output data, (2) the choice of a parametrized model class, and (3) a method for choosing a model from within that class. The classification of a regression problem as *linear* or *nonlinear* depends solely on ingredient (2), the parametrized model class: if the models in that class depend linearly on the model parameters, the regression problem is a linear regression problem.

Check your knowledge: Do you remember what it means for a function to depend linearly on a variable?

The function $f(z;\theta) : \mathbb{R}^d \times \Theta \to \mathbb{R}$ is said to be *linear* in the parameters θ if, for all $a, b \in \mathbb{R}$ and all $\theta_1, \theta_2 \in \Theta$, the following holds: $f(z; a\theta_1 + b\theta_2) = af(z;\theta_1) + bf(z;\theta_2)$. Note that when we say a function is "linear", we have to specify in *what*. That is, we can also say $f(z;\theta)$ is linear in the *inputs* if, for all $a, b \in \mathbb{R}$ and all $z_1, z_2 \in \mathbb{R}^d$, the following

holds: $f(az_1 + bz_2; \theta) = af(z_1; \theta) + bf(z_2; \theta)$.

The classification of regression problems as linear or nonlinear depends **solely** on the dependence of the functions in the parametrized model class on the **parameters**. That is, we can define functions that are *linear* in θ while being *nonlinear* in z.

Exercise

Consider the model classes (Equation 1.1)-(Equation 1.3) introduced previously. Are these model classes linear or nonlinear in the parameters? In the inputs?

2.2 Mathematical problem formulation

We are now going to introduce an *abstract* mathematical problem formulation that can be used to describe many *specific instances* of linear regression problems. This is a theme of the course and throughout computational mathematics and engineering: abstraction using the language of mathematics lets us isolate the core essence of the problem we're solving and develop powerful algorithms that can solve specific applications of those problems across a wide range of disciplines. I'll introduce the abstract formulation first, and follow it up with some specific examples.

Ingredient 1 (the data set): let $\{(z_i, y_i)\}_{i=1}^N$ be a given data set of paired inputs $z_i \in \mathbb{R}^d$ and outputs $y_i \in \mathbb{R}$.

Ingredient 2 (the parametrized model class): let $x : \mathbb{R}^d \to \mathbb{R}^n$ be a function that maps the d-dimensional input to an n-dimensional *feature vector*. For a fixed x, we will consider the following parametrized model class:

$$\mathcal{F}_\beta := \{x(z)^\top \beta : \beta \in \mathbb{R}^n\} \tag{2.1}$$

Recall that Equation 2.1 is read as "$\mathcal{F}_\beta$ is defined to be the set of all functions $f(z; \beta) = x(z)^\top \beta$ for all $\beta \in \mathbb{R}^n$." This is an abstract way to define the model class for *any* linear regression problem, as we will describe in more detail shortly.

Ingredient 3 (the method of choosing the parameters): let β^* be given by

$$\begin{aligned} \beta^* &= \arg\min_{\beta \in \mathbb{R}^n} \frac{1}{N} \sum_{i=1}^N (f(z_i; \beta) - y_i)^2 \\ &= \arg\min_{\beta \in \mathbb{R}^N} \frac{1}{N} \sum_{i=1}^N (x(z_i)^\top \beta - y_i)^2. \end{aligned} \tag{2.2}$$

Then, we define the *learned model* to be $f(z;\beta^*) = x(z)^\top \beta^*$. We call β^* defined this way the "optimal regression parameters" or just the "optimal parameters", because they are the result of solving an optimization problem. Note that the objective of this optimization function is *defined by the data*, and represents the average squared error of the model over the data set.

Taken together, the three ingredients I have defined above define a *linear least squares problem*, which is a subclass of linear regression problems. We'll now give several examples of specific instances of linear least squares problems to illustrate how broadly applicable this abstract framework is.

2.3 Examples

2.3.1 Example 1: Aerodynamic drag prediction

An important task in aerodynamic design is predicting lift and drag forces on a body moving through air. Let's consider a simplified problem where we are given an fixed airfoil design and our goal is to predict the drag force F_d on the airfoil as a function of three parameters which describe its flight conditions:

- The angle of attack α
- The density of the fluid ρ
- The freestream velocity of the air v

To put this problem in our abstract framework, we define $z = (\alpha, \rho, v)^\top$ to be a three-dimensional input, and take the output to be the drag force $y = F_d$. In order to define a regression problem, we require the existence of a data set $\{(z_i, y_i)\}_{i=1}^N$. Note that in this case $z_i = (\alpha_i, \rho_i, v_i)$. We assume that this data set is given.

In linear regression problems, defining the model class amounts to choosing a set of regression *features* by defining x. A simple choice takes the inputs themselves to be features: $x^{(1)}(z) = z = (\alpha, \rho, v)^\top$. This leads to a class of parametrized models with a three-dimensional unknown parameter vector $\beta \in \mathbb{R}^3$. The models in this class have the following form:

$$f^{(1)}(z;\beta) = x^{(1)}(z)^\top \beta = (\alpha, \rho, v)\beta = \beta_1 \alpha + \beta_2 \rho + \beta_3 v.$$

There are many other possible choices. For example, consider $x^{(2)}(z) = (\alpha, \rho, v, \alpha^2, \rho^2, v^2, \alpha\rho, \alpha v, \rho v)^\top$. This leads to a parametrized model class with a *nine*-dimensional unknown parameter vector $\beta \in \mathbb{R}^9$:

$$\begin{aligned} f^{(2)}(z;\beta) &= x^{(2)}(z)^\top \beta = (\alpha, \rho, v, \alpha^2, \rho^2, v^2, \alpha\rho, \alpha v, \rho v)\beta \\ &= \beta_1 \alpha + \beta_2 \rho + \beta_3 v + \beta_4 \alpha^2 + \beta_5 \rho^2 + \beta_6 v^2 + \beta_7 \alpha\rho + \beta_8 \alpha v + \beta_9 \rho v \end{aligned}$$

For either the above choices of features $x^{(1)}(z)$ or $x^{(2)}(z)$, we could then define and solve the minimization Equation 2.2 to find the optimal regression parameters and define our learned model $f^{(1)}(z;\beta^*)$ or $f^{(2)}(z;\beta^*)$.

> **Understanding notation**
>
> Note that we use β to denote the unknown parameters in both $f^{(1)}$ and $f^{(2)}$ above despite β referring to different quantities in the definition of the different functions. This is a common notational shortcut — while we could use the notation $\beta^{(1)} \in \mathbb{R}^3$ and $\beta^{(2)} \in \mathbb{R}^9$ to specify the different β for the different functions, this can be cumbersome if we are considering many different options for the choice of features $x(z)$, and it's standard to just use β, where the definition of β is implied by the context. One of the challenges in learning about machine learning and computational mathematics more generally is getting used to similar notation meaning different things in different contexts. That's one of the things that we'll practice in this course.

2.3.2 Example 2: Monitoring the Maximum Displacement of a Bridge

Least-squares regression is a powerful tool for estimating unknown quantities in many scientific and engineering disciplines beyond Aerospace. Let's instead say we are interested in accurately estimating the maximum displacement at the midpoint of a bridge under various input-conditions measured by sensors along the bridge. Suppose we can accurately gather the following inputs:

- Load applied to the bridge, L
- Temperature at the midpoint of the bridge, T
- Dominant frequency in the bridge, f

We are interested in quickly estimating the Maximum Displacement (mm), D, from these measurements. As we can see, the context of our problem has changed entirely, but we can still fit it into our existing framework by defining the input vector $z = (L, T, f)^\top$ and output $y = D$. If we take N measurements throughout the year under various environmental conditions, we can again form some training dataset $\{(z_i, y_i)\}_{i=1}^N$ which we can use to define some feature-set $x(z)$ and then solve for the optimal weights that map $x(z)$ to y identically to how we did in Example 1.

2.3.3 Example 3: Detecting Cardiac Arrhythmia using Health-Monitoring Devices

Many health issues are often difficult to detect without extensive testing. However, with the increase in biometric data available from devices such as smartphones and smart-watches, it has become increasingly feasible to predict health conditions by leveraging this information. VO_2

max is a measure of the maximum rate at which a person can use oxygen during exercising and is considered a standard measure of cardiovascular fitness. However, to measure this experimentally requires an extensive laboratory test that measures the volume of oxygen a patient breathes during exercising. Let's now suppose we have developed a smart-watch that gathers the following health-data from its user:

- Median daily Step-Count s
- Resting Heart Rate r
- Respiratory Rate b

From these measurements, we are interested in accurately estimating the VO_2 max of a patient. Let's say we have a large sample of many patients of various ages and demographics who have worn our smart-watch for a long period of time. For the i th patient, suppose we have s_i, r_i and b_i collected from the device and we have measured each patient's actual VO_2 max in a laboratory setting, which we denote y_i. We then form a training dataset, $\{(z_i, y_i)\}_{i=1}^N$ and solve the least-squares regression problem using the discrete values for y exactly how we did in Example 1.

2.4 Solving linear least squares problems

Linear least squares problems are special because they have closed form solutions: that is, we can write an analytical expression for the optimum parameters β^* in terms of the data. To do so, we are going to define a feature data matrix $X \in \mathbb{R}^{N\times n}$ and an output data vector $Y \in \mathbb{R}^N$ as follows:

$$X = \begin{pmatrix} - & x^\top(z_1) & - \\ & \vdots & \\ - & x^\top(z_N) & - \end{pmatrix}, \qquad Y = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix},$$

where the *rows* of X and the *elements* of Y correspond to input-output pairs in the data set. Note that each *column* of X corresponds to a different *feature* defined by an element of the vector-valued function x.

Using this notation, Equation 2.2 can be rewritten as

$$\beta^* = \arg\min_{\beta\in\mathbb{R}^n} \frac{1}{N}\|X\beta - Y\|^2 \tag{2.3}$$

This is the minimization of a multivariate function (because β is a vector). Recall from calculus that to find minimizers of multivariate functions we first seek *critical points* that satisfy the *first-order necessary conditions* for optimality: that is, we look for points where the derivative of the objective function is 0. Let $\mathcal{L}(\beta) = \frac{1}{N}\|X\beta - Y\|^2 = \frac{1}{N}(X\beta - Y)^\top(X\beta - Y)$. Then,

$$\frac{\partial \mathcal{L}}{\partial \beta} = \frac{2}{N}(X^\top X\beta - X^\top Y)$$

Setting the derivative equal to 0 yields the standard *normal equations*[1]:

$$X^\top X\beta^* = X^\top Y$$

If $(X^\top X)$ is invertible, then the unique solution to the normal equations is given by

$$\beta^* = (X^\top X)^{-1}X^\top Y$$

and this choice of β defines our final learned model: $f(z;\beta^*) = x(z)^\top\beta^*$.

2.4.1 Using Python for Least-Squares Regression

Now let's explore how we can actually solve for our optimal weights, β^*, given some training data with code. First, we need to import the libraries we need to work with matrices/vectors (`numpy`), plot results (`matplotlib`), work with dataframes (`pandas`), and easily perform more math than base python allows like π, sines, cosines, and square-roots (`math`).

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import math
```

Now let's say we've imported some pandas dataframe called `df` that contains a set of empirical data from Example 1 (Assume it comes from experimental trials or simulations). We can print the first few rows of the dataset using the `df.head()` method:

```
df.head()
```

	alpha	rho	velocity	f_drag
0	16.854305	1.319114	79.481130	9425.204935
1	42.782144	1.203298	50.263709	13722.379248
2	32.939727	1.320528	57.690388	12863.441979

[1] note that the constant factor $\frac{2}{N}$ drops out. It's common to play fast and loose with multiplicative constants in minimizations – other sources define the minimization objective with a $\frac{1}{2}$ multiplier, which leads to the derivative not having the factor of 2 in front and leads to the same critical point. Or you may see the objective function defined without the $1/N$ in front, which still leads to the same critical point.

	alpha	rho	velocity	f_drag
3	26.939632	0.696729	49.251769	3644.076532
4	7.020839	1.227098	19.524299	186.612122

Let's define our input features as $x(z) = (\alpha, \rho, v)^\top$. We can now form X (our training inputs) and Y (our training outputs) matrices by extracting the input features and outputs from the dataframe:

```
# The hstack method concatenates column vectors into
# an N x d matrix like [1 alpha rho velocity]
X = np.hstack((np.ones((N, 1)), df[['alpha', 'rho', 'velocity']].values))
Y = df['f_drag'].values
```

We can solve for β^* using the `np.linalg.lstsq()` function which efficiently solves the least-squares equation $X\beta = Y$. The `rcond` keyword argument specifies how "well-conditioned" we want our result to be, achieved by cutting off the smallest singular values of X (don't worry too much about this for now). Setting it to `None` assumes X is well-conditioned i.e. far from being rank-deficient. By default, the `lstsq()` function returns a list of the optimal weights, β^*, the residuals of the training data ($X\beta - Y$), the rank of X, and the singular values of X. For now, we're only interested in β^*, so we use `[0]` to select this value.

```
beta_star = np.linalg.lstsq(X, Y, rcond=None)[0]
```

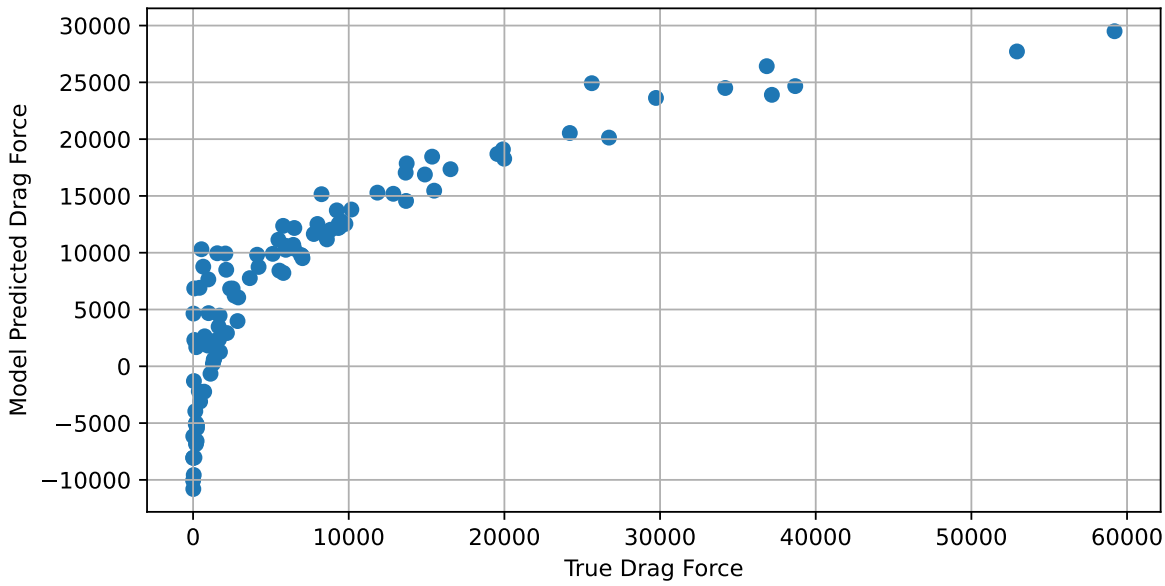
We are now able to compute the model's predictions at the training inputs:

```
Y_hat = X @ beta_star
```

Let's now plot $\hat{Y}$ and Y to examine how closely correlated the two are. A straight line with slope 1 would mean the linear regression model exactly matched all of the training outputs:

```
plt.figure(figsize=(8,4))
plt.scatter(Y, Y_hat)
plt.grid()
plt.xlabel("True Drag Force")
plt.ylabel("Model Predicted Drag Force")
```

```
Text(0, 0.5, 'Model Predicted Drag Force')
```



This plot shows that there is clearly a strong, but nonlinear relationship between the true and model-predicted drag-force. We observe that the linear regression model under-predicts the drag-force when the true force is very low or very high and the model over-predicts the drag-force when the true force is around 5000N. This shows that when the relationship between the input features and the output is nonlinear, linear regression may provide good estimates on average, but choosing features to better capture this nonlinearity may provide much more accurate outputs over the entire input domain.

2.5 Assessing the learned models

2.5.1 Common Performance Metrics

Once we have trained a model (in the linear regression case, we have computed the optimal β that maps our training inputs to our training outputs), it's important to have some quantitative way to measure how "well" our model is able to do this. There are many, many ways to measure model performance. For regression problems, the popular `sklearn` python package for machine learning has thirteen different metrics to assess model performance. We will outline some of the most prominent metrics in this section.

A natural way to assess model performance is to measure the error between the model's predictions and actual outputs. One common way to do this is computed is with *mean squared error*:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the model's best approximation of y_i, using inputs x_i. Because of how linear regression is formulated, the estimator β^* will be the set of weights that minimize the MSE of the training data. If X is full-rank, β^* will be unique. However, many data-scientists dislike the MSE because it penalizes higher errors more than lower errors and in the context of physical experiments the squared error doesn't have easily interpretable units. For instance, in Example 1 we sought to predict drag-force in Newtons. Using the MSE would produce an error with units of (Newtons)^2 which doesn't have an intuitive physical interpretation. For this reason, many scientists use the *mean absolute error*:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|$$

In Example 1, this would give us an error with units of Newtons that we can interpret as "the linear regression model deviates from the true outputs by _____ Newtons on average". If the output data significantly varies in magnitude over the domain of the input, then we may expect MSE and MAE to dominate in places where the magnitude of y is large. To combat this problem, we might choose to use *mean relative error* to assess the performance of our model. This scales the error at each input/output pair according to how large the output is:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\frac{|\hat{y}_i - y_i|}{|y_i|}$$

At the end of **2.4**, we plotted each training output, y_i, on the x-axis of a plot and the model's best guess ($\hat{y}_i$) on the y-axis. If the model was able to perfectly match each y_i, then we should expect a perfect line with a slope of 1 ($\hat{y} = y$). Using the *pearson correlation coefficient*, often denoted R or the *coefficient of determination*, R^2, we can assess how accurately the model is able to match a given set of outputs. We can compute this as:

$$R = \frac{\sum(\hat{y}_i - \bar{\hat{y}})(y_i - \bar{y})}{\sqrt{\sum(\hat{y}_i - \bar{\hat{y}})^2 \sum(y_i - \bar{y})^2}}$$

When $\hat{y}$ perfectly matches the true outputs, y, we get a pearson correlation coefficient of $R = R^2 = 1.0$. Anything lower indicates an imperfect match between the model's predictions and the true outputs.

2.5.2 Cross-Validation

Up to now, we have only discussed model performance as it relates to the data used to train the model. However, the entire reason we go to the trouble of training machine learning models is so they can be used to make accurate predictions on unseen data. Famous LLMs like ChatGPT are so valuable because we can ask them questions they have never seen before and they can still provide coherent, useful answers.

Oftentimes, when machine learning models are highly expressive, (i.e. can model complex functional relationships) they tend to *overfit* to their training data. This happens when the model is so good at matching its training data, it does not generalize well to data it hasn't seen before. This can be especially problematic when our training data contains some sort of noise or uncertainty and our trained model begins to conform to the noise present in the training data.

A straightforward method to assess how much a model is overfitting its training data is to form a `train` and `test` set. Usually around 80% of the training data is used to train the model and about 20% is kept out of the training process and used to evaluate the model's ability to generalize to data it hasn't seen before. A model that generalizes well should exhibit approximately the same performance on its testing data as its training data. When a model shows significantly worse performance on its `test` dataset, this indicates that the model might be overfitting its training data.

Let's refer back to Example 1, when we were trying to accurately predict the force of drag on an airfoil. Say we break up the training data (`X` and `Y`) into (`Xtrain`, `Ytrain`) and (`Xtest`, `Ytest`); one dataset to train the model and one to evaluate it. Suppose we receive the following values for Mean Absolute Error on the training and test predictions:

- `Training MAE: 61.456 N`
- `Testing  MAE: 595.328 N`

This shows that the model exhibits significantly lower error on its training data, and much higher on data it hasn't seen before. This is typically a dead giveaway of overfitting.

2.6 Exercises

1. Recall that the loss-function for least-squares regression is given by $\mathcal{L}(\beta) = \frac{1}{N}||X\beta - Y||_2^2$. Show that the gradient of $\mathcal{L}$ with respect to β is given by $\nabla_\beta \mathcal{L} = \frac{2}{N}\left(X^\top X\beta - X^\top Y\right)$

   Hint: you may find the following identities from the Matrix Cookbook (Petersen and Pedersen 2012) useful. For $v, u \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n\times n}$, the following hold:

   i. $\nabla_v(v^\top A v) = 2Av$
   ii. $\nabla_v(v^\top u) = \nabla_v(u^\top v) = u$

For an extra-thorough learning experience, try deriving the above identities from the definitions of matrix-vector and dot products.

2. Consider a matrix $A \in \mathbb{R}^{n \times m}$. Which of the following are necessary conditions for A to be invertible? Which are sufficient?

   a. A is a square matrix, i.e. $n = m$

   b. The columns of A span $\mathbb{R}^n$
   c. The columns of A are linearly independent
   d. A has no linearly dependent columns
   e. The null-space of A is empty
   f. The only solution of $AX = 0$ is $X = 0$.

3. Show that $X^\top X$ is invertible if and only if X has full column-rank.

4. Explain why β^* is the unique solution to the normal equations if $(X^\top X)$ is invertible. If $(X^\top X)$ is not invertible, describe all solutions to the normal equations.

2.7 Further reading

*Linear Algebra Crash Course (**?@sec-linalg**)*

The Matrix Cookbook (Petersen and Pedersen (2012))

References

Cambridge Dictionary. 2024. "Data Science." 2024. https://dictionary.cambridge.org/dictionary/english/data-science.

Merriam-Webster Dictionary. 2024. "Machine Learning." 2024. https://www.merriam-webster.com/dictionary/machine%20learning.

Petersen, Kaare Brandt, and Michael Syskind Pedersen. 2012. "The Matrix Cookbook." https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf.

A Linear algebra review

Before we dive into the world of machine learning and optimization, it's important that we get comfortable with the fundamental building blocks of mathematics in higher dimensions: Linear Algebra. This course assumes you have seen basic concepts in Linear Algebra, but we'll provide a refresher to get everyone on the same page.

A.1 Scalars, Vectors, and Matrices

A.1.1 Notation

Let's introduce some notation: $\mathbb{R}$ is the set of real numbers, $\mathbb{C}$ is the set of complex numbers, and the operator $\in$ designates that a variable *belongs* to a set. To define a real scalar (i.e. just a number), c, we write:

$$c \in \mathbb{R}$$

You can read this as "the variable c belongs to the set of 1-dimensional real numbers."

To define a real n dimensional vector, $\mathbf{x}$, we write:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n$$

To define a $m \times n$ matrix $\mathbf{A}$ of real-valued entries, we write:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

A.1.2 The Transpose

The "transpose" of a matrix or vector, swaps the rows and columns. In other words, the first row becomes the first column, the second row the second column and so-on:

$$\mathbf{x}^\top = \begin{bmatrix} x_1 & \dots & x_n \end{bmatrix} \in \mathbb{R}^{1\times n}$$

$$\mathbf{A}^\top = \begin{bmatrix} a_{11} & a_{21} & \dots & a_{m1} \\ a_{12} & a_{22} & & a_{m2} \\ \vdots & & \ddots & \vdots \\ a_{1n} & a_{2n} & \dots & a_{mn} \end{bmatrix} \in \mathbb{R}^{n\times m}$$

Transposing the product of matrices reverses their order:

$$(\mathbf{ABCD})^\top = \mathbf{D}^\top\mathbf{C}^\top\mathbf{B}^\top\mathbf{A}^\top$$

A.1.3 Addition & subtraction

Technically, only objects of the same dimensionality can be added or subtracted with one another. So, scalars can only be added to scalars, vectors can only be added to vectors of the same dimension, and matrices can only be added to other matrices with the same numbers of rows and columns. Some software like MATLAB or NumPy might let you add scalars to vectors and matrices and under the hood they multiply that scalar by a vector/matrix of ones to make the dimensions match.

Examples of `numpy` allowing mathematically invalid operations

Mathematically, none of the following are valid operations. However, `numpy` allows us to do them by secretly adding in a vector of ones so that the vector/matrix dimensions are valid. This can be confusing for new programmers.
Adding a scalar to a vector

```python
import numpy as np
# Defining a vector
u = np.array([
  1, 2, 3
])

# Adding a scalar to a vector
print(u + 5)
# What numpy does to make the dimensions match
print(u + 5*np.ones_like(u))
```

```
[6 7 8]
[6 7 8]
```

Adding a row-vector to a column-vector

```python
# Reshaping u to a column-vector
u = u.reshape((3,1))

# Defining a row-vector, v
v = np.array([
  4, 5, 6
]).reshape((1,3))

# Adding the row-vector v to a column-vector u
print(v + u)
# What numpy does to make the dimensions match
print(np.outer(u, np.ones(3)) + np.outer(np.ones(3), v))
```

```
[[5 6 7]
 [6 7 8]
 [7 8 9]]
[[5. 6. 7.]
 [6. 7. 8.]
 [7. 8. 9.]]
```

A.2 Scalar Multiplication

Generally, scalars can multiply by any structure (scalar, vector, and matrix) and do not change its dimension. They only "scale" its value by a certain amount. Hence, multiplying a scalar by

a scalar returns a scalar, multiplying a scalar times a vector returns a vector, and multiplying a scalar by a matrix returns a matrix. Scalar multiplication is also commutative, i.e. $c\mathbf{A} = \mathbf{A}c$ for all scalars c. Consider our scalar, c, from the previous section, another scalar, b, the vector $\mathbf{x} \in \mathbb{R}^n$ and the matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$

A.2.1 Scalar-Scalar Product

$$b \times c = bc \in \mathbb{R}$$

A.2.2 Scalar-Vector Product

$$c\mathbf{x} = \mathbf{x}c = \begin{bmatrix} cx_1 \\ \vdots \\ cx_n \end{bmatrix} \in \mathbb{R}^n$$

A.2.3 Scalar-Matrix Product

$$c\mathbf{A} = \mathbf{A}c = \begin{bmatrix} ca_{11} & ca_{12} & \dots & ca_{1n} \\ ca_{21} & ca_{22} & & ca_{2n} \\ \vdots & & \ddots & \vdots \\ ca_{m1} & ca_{m2} & \dots & ca_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

A.3 Matrix & Vector Multiplication

A.3.1 Vector-Vector Multiplication

To multiply two matrices (or two vectors, or a matrix with a vector), their inner dimensions must always match. Hence, the only valid way to multiply two vectors is by "inner" or "outer" products. Consider two vectors, $\mathbf{u} \in \mathbb{R}^n$ and $x \in \mathbb{R}^n$. An "**Inner Product**" (sometimes called the Dot-Product) is defined as:

$$\mathbf{u}^\top \mathbf{x} = \begin{bmatrix} u_1 & \dots & u_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = u_1x_1 + u_2x_2 + \dots u_nx_n \in \mathbb{R}$$

Some important notes about inner products:

- If the dimensions of $\mathbf{u}$ and $\mathbf{x}$ do not match, we cannot multiply them this way as we need to multiply each entry of both vectors.
- This operation returns a scalar value.

- Inner Products are commutative, i.e. $\mathbf{u}^\top \mathbf{x} = \mathbf{x}^\top \mathbf{u}$, as they are sums of scalar-scalar multiplications which are also commutative.

Recall $\mathbf{x} \in \mathbb{R}^n$. Now consider a new vector $\mathbf{w} \in \mathbb{R}^m$ where $m \neq n$. An "**Outer Product**" is defined as:

$$\mathbf{w}\mathbf{x}^\top = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix} \mathbf{x}^\top = \begin{bmatrix} w_1 \mathbf{x}^\top \\ \vdots \\ w_m \mathbf{x}^\top \end{bmatrix} = \begin{bmatrix} w_1 x_1 & w_1 x_2 & \dots & w_1 x_n \\ w_2 x_1 & w_2 x_2 & & w_2 x_n \\ \vdots & & \ddots & \vdots \\ w_m x_1 & w_m x_2 & \dots & w_m x_n \end{bmatrix} \in \mathbb{R}^{m \times n}$$

Important notes on outer products:

- Outer products return *matrices* with dimensions according to the vectors multiplied.
- As long as $\mathbf{w}$ and $\mathbf{x}$ are vectors, outer products share an inner dimension of 1, which means that any two vectors have an outer product.
- Lastly, this operation is *non-commutative*, meaning $\mathbf{w}\mathbf{x}^\top \neq \mathbf{x}\mathbf{w}^\top$. Rather, these two are transposes of each other.

A.3.2 Matrix-Vector Multiplication

Multiplying a matrix with a vector means the dimension of the vector must equal the number of columns of the matrix. Recall $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{x} \in \mathbb{R}^n$. Note that the columns of $\mathbf{A}$ match the dimension of $\mathbf{x}$, which are both size n. The product $\mathbf{A}\mathbf{x}$ can be written as:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \end{bmatrix} \in \mathbb{R}^m$$

Perhaps a more useful way to visualize this is to break $\mathbf{A}$ down into its constituent columns and show this as a sum of scalar-vector multiplications:

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} | & | & & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \dots & \mathbf{a}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} | \\ \mathbf{a}_1 \\ | \end{bmatrix} x_1 + \begin{bmatrix} | \\ \mathbf{a}_2 \\ | \end{bmatrix} x_2 + \cdots + \begin{bmatrix} | \\ \mathbf{a}_n \\ | \end{bmatrix} x_n \in \mathbb{R}^m$$

where $\mathbf{a}_i$ is the ith column of $\mathbf{A}$. Note that matrix-vector multiplication forms a linear map from one dimensional vector-space to another. In this case, $\mathbf{A}\mathbf{x}$ maps a vector from $\mathbb{R}^n$ to $\mathbb{R}^m$. This linear transformation from one dimensionality to another is a core component of many machine learning algorithms.

A.3.3 Matrix-Matrix Multiplication

Consider two matrices, $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times k}$, where $m \neq k$. Again, the inner dimensions must match to multiply matrices. Because $\mathbf{A}$ multiplied by $\mathbf{B}$ have an inner-dimension of n, they can be multiplied. Similar to matrix-vector multiplication, we can visualize the product of $\mathbf{A}$ and $\mathbf{B}$ by breaking up the columns and rows of $\mathbf{A}$ and $\mathbf{B}$, respectively:

$$\mathbf{AB} = \begin{bmatrix} | & | & & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \dots & \mathbf{a}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} - & \mathbf{b}_1 & - \\ - & \mathbf{b}_2 & - \\ & \vdots & \\ - & \mathbf{b}_n & - \end{bmatrix} \in \mathbb{R}^{m \times k}$$

where $\mathbf{a}_i$ is the ith column of $\mathbf{A}$ and $\mathbf{b}_i$ is the ith row of $\mathbf{B}$. We can expand this expression by multiplying each column of $\mathbf{A}$ with each row of $\mathbf{B}$:

$$\mathbf{AB} = \begin{bmatrix} | \\ \mathbf{a}_1 \\ | \end{bmatrix} \begin{bmatrix} - & \mathbf{b}_1 & - \end{bmatrix} + \begin{bmatrix} | \\ \mathbf{a}_2 \\ | \end{bmatrix} \begin{bmatrix} - & \mathbf{b}_2 & - \end{bmatrix} + \cdots + \begin{bmatrix} | \\ \mathbf{a}_n \\ | \end{bmatrix} \begin{bmatrix} - & \mathbf{b}_n & - \end{bmatrix}$$

As we can see, this is a sum of outer products (vectors multiplied by transposed vectors). We know $\mathbf{A}$ has m rows and $\mathbf{B}$ has k columns, so each outer product will form a *matrix* of dimension $m \times k$:

$$\mathbf{AB} = \sum_{i=1}^{n} \mathbf{a}_i \mathbf{b}_i \in \mathbb{R}^{m \times k}$$

Note: matrix multiplication is generally non-commutative; The product $\mathbf{AB}$ is valid, because the number of columns of $\mathbf{A}$ matches the number of rows of $\mathbf{B}$. However, the product $\mathbf{BA}$ is not valid, because the number of columns of $\mathbf{B}$ does not equal the number of rows of $\mathbf{A}$. The product $\mathbf{BA}$ can be multiplied if $\mathbf{A}$ and $\mathbf{B}$ are both square matrices of the same size. However, even if the dimensions do match, generally speaking, $\mathbf{BA} \neq \mathbf{AB}$.

A.4 Vector Norms

Oftentimes, it is useful to know how "big" a vector is. And there are many different ways of computing this. The "norm" of an object is a measure of how "large" it is, according to some rule. Consider $\mathbf{x} \in \mathbb{R}^n$. We'll start with the most common norm:

A.4.1 The 2-Norm

The 2-norm of x (which we will use most-often in this class) is defined as the squareroot of the squares of its entries:

$$||\mathbf{x}||_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

The reason the 2-norm is so useful is the fact that the square of the l_2 norm is an inner product of a vector with itself:

$$\mathbf{x}^\top \mathbf{x} = x_1^2 + x_2^2 + \cdots + x_3^2 = ||\mathbf{x}||_2^2$$

This helps us write a norm in terms of matrix-vector operations, which is extremely useful for optimization. The 2-norm is also quite important because it is the first l_p norm with a continuous derivative, which cannot be said of the 1-norm since it uses absolute-value functions.

A.4.2 The l_p-Norm

We can take the definition of the 2-Norm and generalize it to describe a class of norms known as l_p norms. The l_p norm of a vector is defined by the following:

$$||\mathbf{x}||_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$$

A.4.3 The 1-Norm

The 1-norm of $\mathbf{x}$ is simply the sum of the absolute-value of its entries:

$$||\mathbf{x}||_1 = |x_1| + |x_2| + \cdots + |x_n|$$

A.4.4 The Infinity-Norm

The infinity-norm takes the limit as $p \to \infty$ and when this limit is solved, this norm simply becomes the entry of x with the largest absolute value:

$$||\mathbf{x}||_\infty = \max\{|x_1|, |x_2|, \ldots, |x_n|\}$$

A.5 Rank, Range/Column-Space and Null-Spaces

A.5.1 Linear Combinations and Span

Many of the following ideas hinge upon the idea of "Linear Combinations" of vectors. Consider two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$. A linear combination of $\mathbf{u}$ and $\mathbf{v}$ is the vector produced by the following:

$$\mathbf{w} = c_1\mathbf{u} + c_2\mathbf{v}$$

where c_1 and c_2 are scalars. We can generalize this idea by saying that all possible realizations of $\mathbf{w}$ live in the vector-space $\mathcal{W}$ defined by the "span" by $\mathbf{u}$ and $\mathbf{v}$. This is denoted:

$$\mathbf{w} \in \mathcal{W} = \text{Span}\left(\mathbf{u}, \mathbf{v}\right)$$

We can read this in plain english as "the vector $\mathbf{w}$ belongs to the vector-space $\mathcal{W}$ which is defined by all possible linear combinations of $\mathbf{u}$ and $\mathbf{v}$".

A.5.2 Linear-Independence

This is a fundamental idea in Linear Algebra. A set of vectors is said to be "Linearly Independent" if no vector in this set can be reconstructed with any of the others. In other words, no one vector can be reproduced by scaling or combining any of the others. For example, consider the following three vectors:

$$\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \}$$

Because each vector in this set points in its own unique direction, it is impossible to reconstruct one of these vectors as a linear combination of any others. This may be true with more complex vectors that aren't unit vectors along the dimensions of the vector-space.

A.5.3 Rank

Technically, there are two types of rank: Column Rank and Row-Rank. In this class, when we say "rank", we will be referring to the column rank.

The column rank of a matrix is simply its number of linearly independent column-vectors (the row-rank is the number of linearly independent row-vectors). Consider the following matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 & 4 \\ 0 & 1 & 1 & 2 \\ 0 & 0 & 2 & 0 \end{bmatrix}$$

The first-three columns of $\mathbf{A}$ are linearly independent because they cannot be reproduced using linear combinations of the other columns. However, the last column is a multiple of column 2, which means it *can* be reproduced using a linear combination of columns. Hence, this matrix has a rank of 3.

Theorem: No matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ where $m < n$ can be full-rank. This is because only m vectors can span $\mathbb{R}^m$, and since there are more than m columns, we must have some redundancy.

A.5.4 Range/Column-Space

The Range, Column-Space or Image of a matrix, $\mathbf{A}$, is the span of all possible combinations of its columns. In human words, it describes the space of $\mathbf{Ax}$ for all possible $\mathbf{x}$. Consider the following matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 6 \\ 2 & 4 \end{bmatrix}$$

Note that the second column is a multiple of the first column, hence, the range of $\mathbf{A}$ is:

$$\text{Range}(\mathbf{A}) = \text{span}\left(\begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} \right)$$

A.5.5 Null-Space

The Null-Space of a matrix, $\mathbf{A}$ is the set of all nonzero vectors, $\mathbf{x}$ such that $\mathbf{Ax} = \mathbf{0}$. Consider a simple example:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix}$$

We can find the set of $\mathbf{x}$ that produce the zero-vector by solving the following equation:

$$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This produces the systems of equations:

$$x_1 - x_3 = 0$$
$$x_2 + x_3 = 0$$
$$x_3 = x_3$$

If we isolate each entry of $\mathbf{x}$:

$$x_1 = x_3$$
$$x_2 = -x_3$$
$$x_3 = x_3$$

Hence, we see that x_3 can be any scalar and satisfy this systems of equations:

$$\mathbf{x} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} x_3 = \text{span}\left(\begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \right) = \text{Null}(\mathbf{A})$$

A.5.6 The Identity Matrix

The identity matrix is a square, n-dimensional matrix consisting of ones on its diagonal and zeros elsewhere. It is called the identity matrix because it preserves the identity when multiplied by matrices and vectors.

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

Consider $\mathbf{I} \in \mathbb{R}^{n \times n}$ to be an $n \times n$ identity matrix. If we recall $\mathbf{A} \in \mathbb{m} \times \mathbb{n}$ and $\mathbf{x} \in \mathbb{R}^n$:

$$\mathbf{AI} = \mathbf{IA} = \mathbf{A}$$

$$\mathbf{Ix} = \mathbf{x}$$

$$\mathbf{x}^\top \mathbf{I} = \mathbf{x}^\top$$

> **Why is the identity matrix useful?**
>
> The identity matrix simplifies expressions involving matrices. If we can transform a term into an identity matrix, then we can reduce the expression to only the remaining terms.

A.5.7 Matrix Inverses

You may have noticed that, thus far, we have covered the addition, subtraction, and multiplication of matrices, which conspicuously leaves division. Well, unfortunately, matrix division is complex to say the least. For square matrices, division is accomplished via matrix inversion. However, not all matrices are invertible.

Definition: A matrix $\mathbf{B}$ is called the "inverse" of a square matrix $\mathbf{A}$ if and only if the following condition holds:

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I}$$

where $\mathbf{I}$ is the identity matrix. Such a $\mathbf{B}$ is denoted $\mathbf{A}^{-1}$.

A.6 Further Reading

- A great read for a deeper dive into the relationship between Column, Row and Null-spaces of matrices: Gil Strang's Fundamental Theorem of Linear Algebra